

Scalable Distributed Computing - 2026/27

Lecture 7 - Performance Measurement and Empirical Performance Analysis

Patrizio Dazzi patrizio.dazzi@unipi.it

ACT I

Observing system performance

L6 characterized the input workload; L7 characterizes the observed system response.

Throughput measures completed work

Throughput describes how much work the system completes per unit time.

$$X = \frac{\text{completed operations}}{\Delta t}.$$

For example,

$$X = 8{,}000 \text{ requests/s}.$$

Throughput refers to **completed work**, not merely work offered to the system.

Arrival rate describes input; throughput describes completed output.

Throughput can be defined from cumulative completions

Let

$$C(t) = \text{number of operations completed up to time } t.$$

During the interval $[t, t+T]$, the number of completions is

$$C(t+T) - C(t).$$

Therefore the average throughput over that window is

$$\boxed{\overline{X}_T(t) = \frac{C(t+T) - C(t)}{T}}.$$

Throughput is a completion rate measured over an explicitly defined time window.

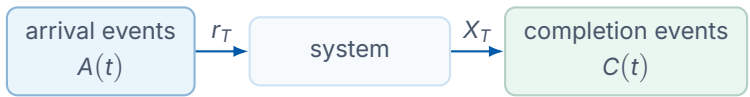
Arrivals and completions use the same measurement grammar

Define cumulative counts

$$A(t) = \text{arrivals up to } t, \qquad C(t) = \text{completions up to } t.$$

Over the same window $[t, t+T]$,

$$r_T(t) = \frac{A(t+T) - A(t)}{T}, \qquad X_T(t) = \frac{C(t+T) - C(t)}{T}.$$



The equations have the same form; the counted events answer different questions.

Worked example: compute throughput from completion counts

Suppose

$$C(10) = 120{,}000, \qquad C(16) = 168{,}000.$$

The number of completions during the six-second window is

$$C(16) - C(10) = 168{,}000 - 120{,}000 = 48{,}000.$$

Hence

$$\overline{X}_6(10) = \frac{48{,}000}{16 - 10} = \frac{48{,}000}{6} = \boxed{8{,}000 \text{ requests/s}}.$$

The numerator is completed work; the denominator is the duration over which that work was observed.

Latency measures the time experienced by one operation

For a single request,

$$L = t_{\text{completion}} - t_{\text{arrival}}.$$

If

$$t_{\text{arrival}} = 12.000\ \text{s}, \qquad t_{\text{completion}} = 12.080\ \text{s},$$

then

$$L = 12.080 - 12.000 = 0.080\ \text{s} = \boxed{80\ \text{ms}}.$$

Throughput describes aggregate progress; latency describes one operation's elapsed time.

Throughput and latency describe different views

Throughput

$8{,}000$ requests/s

asks:

How much work is completed?

Latency

80 ms

asks:

How long does one operation take?

A system can have high throughput while some individual requests still complete slowly.

No single metric fully describes performance.

Latency is not usually a single number

Suppose five requests complete in

$$12,\ 13,\ 14,\ 15,\ 90 \text{ ms}.$$

The requests experienced very different latencies.

Reporting only one value can hide this variation.

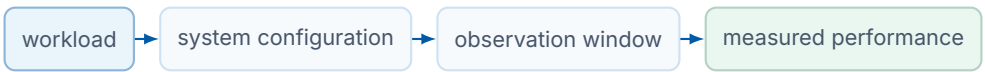
Latency should be treated as a distribution of observations, not only as one summary value.

Performance measurements always refer to an interval

Suppose the system completes $48{,}000$ requests during a six-second observation window. Then

$$X = \frac{48{,}000}{6} = 8{,}000 \text{ requests/s.}$$

Another interval, even under nominally similar conditions, may produce a different value.



A measured performance value belongs to a specific workload, configuration, and observation interval.

A performance observation needs context

A meaningful measurement should identify at least

workload system configuration measurement interval

and the observed quantities, such as

throughput latency.

A number without its measurement context is not a performance result.

ACT II

From latency samples to a distribution

A single request has a latency; a workload produces many latency observations.

A workload produces a latency distribution

During one measurement interval, suppose we observe

$$12,\ 14,\ 13,\ 18,\ 15,\ 16,\ 42,\ 14,\ 13,\ 75 \quad \text{ms}.$$

There is no single “latency of the system” in these data.
Instead, we have a collection of observations:

$$L_1, L_2, \ldots, L_n.$$

Latency is naturally described by a distribution of observations.

The mean summarizes the center, but not the shape

Consider two latency samples:

$$A = \{18, 19, 20, 21, 22\} \text{ ms}$$

$$B = \{5, 10, 15, 20, 50\} \text{ ms}.$$

Both have

$$\overline{L} = 20 \text{ ms}.$$

But the request experiences are clearly different.

Different latency distributions can have the same mean.

Ordering the observations exposes their spread

Take the sample

$$12,\ 14,\ 13,\ 18,\ 15,\ 16,\ 42,\ 14,\ 13,\ 75 \text{ ms}.$$

After sorting,

$$12,\ 13,\ 13,\ 14,\ 14,\ 15,\ 16,\ 18,\ 42,\ 75.$$



Ordering reveals spread and slow observations that a single average discards.

The median asks for the middle observation

For an ordered sample, the median separates the observations into two halves.

For

$$12,\ 13,\ 13,\ 14,\ 14,\ 15,\ 16,\ 18,\ 42,\ 75,$$

the middle lies between 14 and 15 ms, so

$$\text{median} = 14.5 \text{ ms}.$$

The large values 42 and 75 influence the mean much more strongly than the median.

Mean and median summarize different properties of the same distribution.

From latency samples to an empirical distribution

For latency observations $L_1, \ldots, L_n$, define the empirical cumulative distribution

$$\widehat{F}_L(\ell) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}[L_i \leq \ell].$$

Here $\mathbf{1}[L_i \leq \ell]$ is 1 when request i completed within ℓ, and 0 otherwise.

Thus $\widehat{F}_L(\ell)$ is simply the observed fraction of requests whose latency is no greater than ℓ.

The empirical distribution turns a collection of latency samples into a quantitative performance description.

Worked example: evaluate one empirical-CDF point

For the ordered latency sample

$$12, 13, 13, 14, 14, 15, 16, 18, 42, 75 \text{ ms},$$

consider the threshold $\ell = 18$ ms.
Eight of ten observations satisfy $L_i \leq 18$ ms, hence

$$\widehat{F}_L(18) = \frac{1}{10} \sum_{i=1}^{10} \mathbf{1}[L_i \leq 18] = \frac{8}{10} = 0.8.$$

$\widehat{F}_L(18) = 0.8$ **means that 80% of the measured requests completed within 18 ms.**

Worked example: watch the empirical CDF step upward

For the same sample,

ℓ	$\#\{L_i \le \ell\}$	$\widehat{F}_L(\ell)$
18 ms	8	0.8
42 ms	9	0.9
75 ms	10	1.0

The empirical CDF changes only when the threshold crosses an observed latency.

In this sample there is no empirical probability level between 0.9 and 1.0: the last observation creates one large final step.

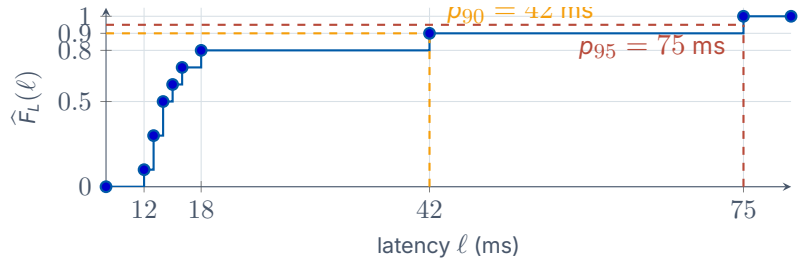
A finite sample produces a stepwise empirical distribution.

The empirical CDF makes percentiles visible

For the sample

$$12, 13, 13, 14, 14, 15, 16, 18, 42, 75 \text{ ms},$$

its empirical CDF is a step function.



A percentile is read by moving from a probability level to the smallest latency where the ECDF reaches it.

Percentiles describe positions in the empirical distribution

Using the empirical distribution, the q-quantile can be written as

$$\boxed{p_q = \inf\{\ell : \widehat{F}_L(\ell) \geq q\}.}$$

For example,

p_{50} is the median, p_{95} describes the upper part of the latency distribution.

Conceptually,

$$p_{95} = L$$

means that approximately 95% of the observed requests had latency no greater than L.

Modelling note. Exact sample-percentile estimators differ slightly in how they interpolate between observations. We do not need that detail here.

Worked example: locate p_{90}

For the sample just analysed,

$$\widehat{F}_L(18) = 0.8, \qquad \widehat{F}_L(42) = 0.9.$$

Using

$$p_q = \inf\{\ell : \widehat{F}_L(\ell) \geq q\},$$

p_{90} is the smallest latency threshold whose empirical CDF reaches 0.90. Therefore

$$\boxed{p_{90} = 42 \text{ ms}}.$$

A percentile is obtained by inverting the empirical CDF at the desired probability level.

Worked example: why does p_{95} jump to 75 ms?

For the same ten-request sample,

$$\widehat{F}_L(42) = 0.90 < 0.95, \qquad \widehat{F}_L(75) = 1.00 \geq 0.95.$$

Therefore

$$\boxed{p_{95} = \inf\{\ell : \widehat{F}_L(\ell) \geq 0.95\} = 75 \text{ ms}}.$$

Because $n = 10$, every empirical step has height

$$\frac{1}{n} = 0.10.$$

Modelling note. With ten observations, the empirical CDF jumps from 0.90 directly to 1.00, so p_{95} cannot be resolved more finely.

Upper percentiles reveal slow requests hidden by the center

Suppose two services both report

$$\text{median latency} = 20 \text{ ms}.$$

Service A

slower requests remain around 25 ms

Service B

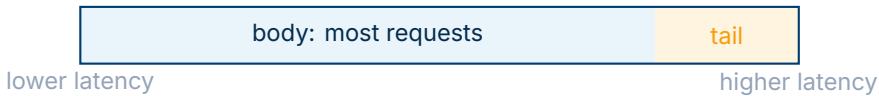
slower requests extend toward 250 ms

The median cannot distinguish these tails. Upper percentiles such as p_{95} or p_{99} probe progressively farther into the upper part of the distribution.

Central and upper-percentile latencies answer different performance questions.

The upper end of the distribution is the latency tail

Informally, the **tail** is the relatively small fraction of requests with the largest latencies.



A workload may therefore exhibit **good typical latency** while a smaller set of requests experiences **poor tail latency**.

The typical request does not necessarily represent every request.

Throughput and latency should be interpreted together

Suppose two runs both achieve

$$X = 10{,}000 \text{ requests/s.}$$

But:

	Run A	Run B
p_{50}	18 ms	20 ms
p_{95}	25 ms	140 ms

The throughput is the same.
The observed request-level performance is not.

A throughput value alone does not characterize the quality of completed work.

ACT III

Performance under increasing load

A system response is meaningful only relative to the workload level at which it is observed.

One measurement gives one operating point

Suppose we observe

$$r_{\text{in}} = 2{,}000 \text{ requests/s}$$

and measure

$$X = 1{,}980 \text{ requests/s}, \qquad p_{95} = 22 \text{ ms}.$$

This tells us how the system behaved at that workload level.

It does not tell us what happens at

$$3{,}000,\ 5{,}000,\ 8{,}000 \text{ requests/s}.$$

Performance is a function of operating conditions, not a single number.

A performance curve comes from repeated operating points

We can deliberately evaluate the system at increasing workload levels:

$$r_1,\ r_2,\ r_3, \ldots$$

and measure, for each one,

$$X(r_i) \qquad \text{and} \qquad L(r_i).$$

This produces two complementary views:

$$\boxed{\text{workload} \longrightarrow \text{throughput}}$$

$$\boxed{\text{workload} \longrightarrow \text{latency}}$$

A curve reveals behavior that a single operating point cannot.

Offered work and completed work may diverge

At low workload, we may observe $r_{\text{in}} \approx X$. As offered load increases, this need not remain true.

r_{in}	X
2,000	1,980
4,000	3,950
6,000	5,700
8,000	5,850

The final increase adds $2{,}000$ offered requests/s but only 150 completed requests/s.

Increasing offered load does not guarantee a proportional increase in completed throughput.

The throughput curve is only part of the picture

For the same workload levels:

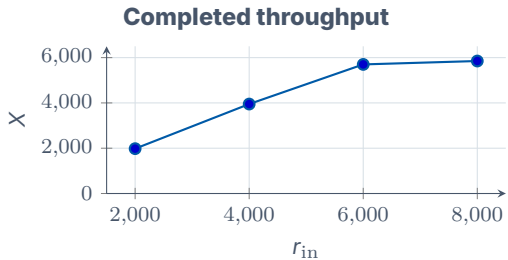
r_{in}	X	p_{95}
2,000	1,980	22 ms
4,000	3,950	28 ms
6,000	5,700	55 ms
8,000	5,850	210 ms

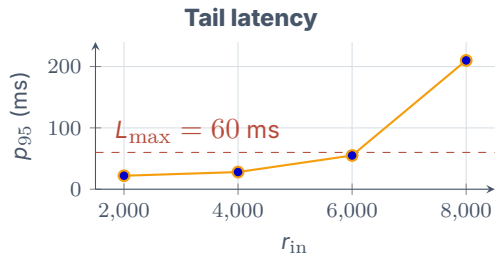
The final workload increase produces little additional throughput, but a large latency increase.

Throughput gains can flatten while latency deteriorates sharply.

One load sweep produces two complementary curves

For the same measured operating points:





The last load increase adds little throughput while moving the system far outside the latency-acceptable region.

Finite differences expose diminishing throughput gains

Between two measured load levels, use the discrete marginal gain

$$\frac{\Delta X}{\Delta r_{\text{in}}}.$$

interval	ΔX	$\Delta X/\Delta r_{\text{in}}$	Δp_{95}
$2000 \to 4000$	1970	0.985	6 ms
$4000 \to 6000$	1750	0.875	27 ms
$6000 \to 8000$	150	0.075	155 ms

For the first interval,

$$\frac{3950 - 1980}{4000 - 2000} = 0.985.$$

Marginal throughput gain collapses while the latency penalty grows sharply.

Highest observed throughput is not necessarily usable capacity

Among the measured operating points, the largest observed throughput is

$$X_{\text{obs,max}} = 5{,}850 \text{ requests/s.}$$

But that point has

$$p_{95} = 210 \text{ ms.}$$

If such latency is unacceptable for the application, that operating point is not usable even though it has the highest measured throughput.

“Highest measured throughput” and “acceptable operating point” answer different questions.

Usable capacity can be formulated as a constrained measurement problem

Suppose the acceptable region is defined by

$$p_{95}(r) \leq L_{\max}, \qquad L_{\max} = 60 \text{ ms}.$$

For the workload levels actually measured, define

$$\boxed{r_{\text{usable}}^{\text{meas}} = \max\{r_i : \; p_{95}(r_i) \leq L_{\max}\}.}$$

Here $6{,}000$ requests/s satisfies the constraint and $8{,}000$ does not, hence

$$r_{\text{usable}}^{\text{meas}} = 6{,}000 \text{ requests/s}.$$

Modelling note. The true boundary may lie between $6{,}000$ and $8{,}000$ requests/s. Finding it requires additional measurements or an explicit model.

Worked example: classify the measured operating points

With the acceptance constraint

$$p_{95}(r_i) \leq 60 \text{ ms},$$

classify each measured point:

r_i	$p_{95}(r_i)$	constraint satisfied?
2,000	22 ms	✓
4,000	28 ms	✓
6,000	55 ms	✓
8,000	210 ms	×

Thus the measured feasible set is

$$\boxed{\mathcal{F}_{\text{meas}} = \{2000, 4000, 6000\}}.$$

Worked example: take the maximum feasible measured load

From the previous classification,

$$\mathcal{F}_{\text{meas}} = \{2000, 4000, 6000\}.$$

Therefore

$$\boxed{r_{\text{usable}}^{\text{meas}} = \max \mathcal{F}_{\text{meas}} = 6000 \text{ requests/s}}.$$

The next measured point, 8000 requests/s, violates the latency constraint. So the exact crossing is not measured; we only know that it lies between the last feasible and first infeasible tested loads:

$$\boxed{6000 \leq r_{\text{boundary}} < 8000}.$$

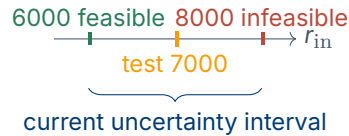
The best measured feasible point is a bound on the unknown boundary, not its exact value.

Additional measurements can refine an unknown capacity boundary

From the tested points we know only

$$6000 \leq r_{\text{boundary}} < 8000.$$

A new experiment at the midpoint can reduce that uncertainty.



Suppose the new run gives $p_{95}(7000) = 85$ ms. Then

$$6000 \leq r_{\text{boundary}} < 7000.$$

A measured capacity limit can be refined experimentally without pretending that an unmeasured point is known.

But can we trust the curve?

A performance curve is only meaningful if its points are comparable.
Before interpreting it, we must ask:

Was the workload defined consistently?

Were the measurement intervals comparable?

Did the system reach a stable operating condition?

The next problem is not modelling the curve, but measuring it correctly.

ACT IV

Measuring performance correctly

Comparable operating points require a consistent measurement procedure.

Do not measure every moment in the same way

A performance run often contains different phases:



During start-up, caches, connections, runtimes, buffers, and other system state may still be evolving.

Define explicitly when measurement begins and ends.

Warm-up must be justified by observed behavior

It is tempting to say:

"ignore the first 30 seconds."

But why 30?

Instead, inspect the evolution of relevant measurements over time.

time $\longrightarrow$ observed performance

The purpose is to identify whether the system is still undergoing a systematic transition.

Discarding an initial interval is justified by system behavior, not by convention.

A stable operating region need not be perfectly constant

A system under a fixed workload may still exhibit variability.

We are not looking for

$$X(t) = \text{constant}.$$

We are looking for a region without an obvious systematic transition such as

continuous growth, continuous decline, unfinished start-up.

Variability and instability are not the same thing.

A measurement window can be too short

Suppose throughput varies during a run.
A 2-second interval may observe

$$X = 8{,}700 \text{ requests/s},$$

while a longer interval over the same operating condition gives

$$X = 8{,}050 \text{ requests/s}.$$

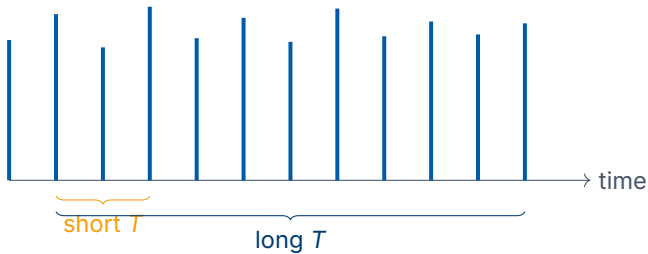
Short windows are more exposed to transient variation.

The observation interval must be long enough to represent the behavior being studied.

A measurement window acts as a temporal filter

The throughput definition already contains the averaging window:

$$X_T(t) = \frac{C(t+T) - C(t)}{T}.$$



Short T preserves fast variation; longer T averages more of that variation away.

Modelling note. Choose T to match the behavior the experiment is intended to characterize.

Repeat the same operating point

Suppose the same experiment produces

$$8{,}120, \quad 7{,}940, \quad 8{,}080, \quad 7{,}990, \quad 8{,}110$$

requests/s across five runs.

The variation itself is information.
A single run would hide it.

Repeated runs reveal whether an observed result is reproducible.

Repeated runs can be summarized quantitatively

For repeated measurements $X_1, \ldots, X_m$,

$$\overline{X} = \frac{1}{m}\sum_{j=1}^{m} X_j, \qquad s_X = \sqrt{\frac{1}{m-1}\sum_{j=1}^{m}(X_j - \overline{X})^2}.$$

The first quantity summarizes the center of the repeated measurements; the second summarizes their run-to-run spread.

For the five runs just shown,

$$\boxed{\overline{X} = 8{,}048 \text{ requests/s}, \qquad s_X \approx 79 \text{ requests/s}}.$$

Report repeatability quantitatively rather than relying on a single run.

Worked example: compute deviations from the repeated-run mean

For the five runs,

$$x_j \in \{8120, 7940, 8080, 7990, 8110\}, \qquad \overline{x} = 8048.$$

The deviations are

$$x_j - \overline{x} \in \{72, -108, 32, -58, 62\}.$$

Squaring them gives

$$5184,\ 11664,\ 1024,\ 3364,\ 3844,$$

whose sum is

$$\boxed{\sum_{j=1}^{5} (x_j - \overline{x})^2 = 25{,}080}.$$

Worked example: compute standard deviation and relative spread

The sample standard deviation is

$$s_X = \sqrt{\frac{1}{5-1}\sum_{j=1}^{5}(X_j - \overline{X})^2}.$$

Substituting the previous result,

$$s_X = \sqrt{\frac{25{,}080}{4}} = \sqrt{6{,}270} \approx 79.2 \text{ requests/s}.$$

Relative to the mean,

$$\boxed{\frac{s_X}{\overline{X}} = \frac{79.2}{8048} \approx 0.0098 \approx 0.98\%}.$$

Absolute and relative spread provide complementary views of repeatability.

Aggregation can hide structure

Suppose two runs have the same average throughput:

$$\overline{X} = 8{,}000 \text{ requests/s.}$$

One is approximately stable.
The other alternates between

$$5{,}000 \quad \text{and} \quad 11{,}000 \text{ requests/s.}$$

The average alone cannot distinguish them.

Inspect temporal behavior before reducing measurements to summary statistics.

Do not remove inconvenient measurements

Suppose one run produces an unexpected point.
There are at least two possibilities:

measurement problem or real system behavior.

Removing the point because it “looks wrong” destroys evidence.
Instead, investigate whether there is a documented reason to exclude it.

Data cleaning is part of the methodology and must be explainable.

A single experiment may cross different operating regimes

As workload increases, a previously irrelevant limitation may become active.
Then measurements may show an abrupt change in behavior.

regime A ⟶ change ⟶ regime B.

This is not automatically noise.
It may indicate that the system is now operating under different constraints.

Do not force one interpretation across an observed change of regime.

ACT V

From question to evidence

A performance result is only as meaningful as the path that produced it.

Turn a performance statement into a claim with evidence

Compare two statements:

Vague

"The service can handle about 6000 requests/s."

Evidence-scoped

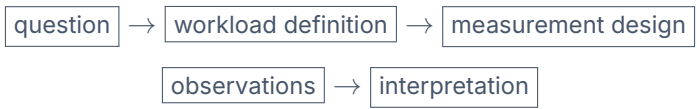
Under workload W and configuration S, the tested point $r_{\text{in}} = 6000$ requests/s satisfies $p_{95} \leq 60$ ms across the specified measurement procedure.

A useful result records the chain

claim ⟵ observations ⟵ measurement design ⟵ workload + configuration.

The strength of a conclusion cannot exceed the scope of the evidence that supports it.

From a performance question to an interpretable result



At every step, ambiguity propagates forward.

Good performance analysis begins before the first measurement is collected.

What should a performance result report?

At minimum:

workload | system configuration

measurement interval | performance metrics

And, when relevant:

generation method | repetitions | data treatment.

A performance number without its context is not reproducible evidence.

Do not confuse three different questions

1. What workload is presented to the system?
2. How does the system respond?
3. Why does the system behave that way?

This lecture has addressed the first two.
The third requires additional models and system knowledge.

Measurement describes behavior; explanation requires a model.

Models and measurements play different roles

A model lets us simplify a system enough to reason about it. A measurement tells us whether the resulting claim is supported by evidence.

When we use a model to interpret measurements, keep four questions visible:

- What does it preserve?
- What does it discard?
- What can I derive?
- How do I validate it?

observe → abstract → model → derive → measure → revise

A model is an instrument for reasoning, not a substitute for evidence.

What we can now do

We can now:

- measure throughput and latency without confusing input and output rates;
- describe latency empirically through distributions and percentiles;
- interpret throughput and tail latency together across load levels;
- identify measured feasible operating points under explicit constraints;
- design measurement windows and repeated runs that support reproducible evidence;
- avoid turning measurements into explanations without an explicit model.

A later step is to explain how load, waiting, and limited service capacity interact.

Selected references for further study

- D. J. Lilja, *Measuring Computer Performance: A Practitioner's Guide*, Cambridge University Press, 2000.
- R. Jain, *The Art of Computer Systems Performance Analysis*, Wiley, 1991.
- M. Harchol-Balter, *Performance Modeling and Design of Computer Systems*, Cambridge University Press, 2013.
- J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, 56(2), 74–80, 2013.

Performance evidence starts with careful measurement and uses models only with explicit assumptions and scope.

Q & A

Any questions?